RAIT: RETRIEVAL-AUGMENTED INVARIANT TRANSFER — WHEN SHARING VERIFIED KNOWLEDGE HELPS (AND WHEN IT DOESN'T)

Anonymous authors
Paper under double-blind review

ABSTRACT

Loop invariant generation is a central challenge in automated program verification, and recent LLM-based approaches treat each program in isolation, discarding rich structural patterns shared across programs. We introduce RAIT (Retrieval-Augmented Invariant Transfer), a framework that builds a self-growing library of verified (program, invariant) pairs and uses structure-aware retrieval to accelerate invariant generation for new programs. Through a controlled simulation study, we evaluate five retrieval strategies — zero-shot, random, semantic, confidence-weighted, and difficulty-adaptive — across MBPP, HumanEval, and CodeSearchNet benchmarks. Our key finding is that retrieval-augmented adaptation consistently improves the Invariant Transfer Efficiency Score (ITES) over zero-shot generation, with confidence-weighted semantic retrieval achieving the best balance of success rate and query efficiency. However, library growth yields non-monotonic efficiency scaling, and mixing confidence scores into retrieval ranking can *degrade* performance relative to pure semantic similarity. These inconclusive and sometimes counterintuitive results highlight real pitfalls in applying retrieval-augmented generation to formal verification, motivating further investigation into library curation, retrieval diversity, and adaptation quality.

1 INTRODUCTION

Loop invariants are logical assertions that hold at every iteration of a loop and are essential for formal program verification. Generating them automatically is notoriously difficult: the space of candidate invariants is infinite, and correctness must be verified by an SMT solver or model checker. Recent work combining LLMs with SMT solvers (**???**) has achieved strong results on standard benchmarks such as Code2Inv (**?**), but all these approaches share a fundamental limitation: they generate invariants *from scratch* for each new program, ignoring structural patterns shared across programs in the same domain.

We propose RAIT (Retrieval-Augmented Invariant Transfer), which frames loop invariant generation as a retrieval-and-adaptation problem. Given a target program, RAIT retrieves the k most structurally similar programs from a growing library of verified (program, invariant) pairs, presents their invariants as context to an LLM, and prompts adaptation to the target. Successfully verified programs are added back to the library, creating a self-improving system. The central hypothesis is appealing: structurally similar programs should share similar invariant shapes, so retrieval should reduce the adaptation burden and the number of LLM queries needed. However, our simulation study reveals that the story is more nuanced. While retrieval-augmented strategies outperform zero-shot generation on efficiency metrics, library growth does not monotonically improve performance, and confidence-based reranking can hurt rather than help.

Contributions: (1) We introduce the RAIT framework for retrieval-augmented loop invariant transfer with a self-growing verified library. (2) We define the Invariant Transfer Efficiency Score (ITES) to jointly capture success rate and query efficiency. (3) Through a controlled simulation study across three code benchmarks, we identify conditions under which retrieval helps and when it introduces

noise. (4) We highlight pitfalls including non-monotonic library scaling, confidence-weight sensitivity, and adversarial library contamination.

2 Related Work

LLM-based loop invariant generation. Early neural approaches (**??**) used specialized architectures to learn invariants from execution traces. More recent work leverages LLMs with SMT feedback: **?** combines reasoning-optimized LLMs with Z3; **?** handles complex programs with data structures; **?** coordinates symbolic execution with LLMs; and BALI (**?**) adds branch-aware static analysis. All these approaches treat each program independently without exploiting shared structure. RAIT is the first to frame this as a retrieval problem. SpecGen (**?**) generates formal specifications using LLMs but does not perform cross-program transfer or maintain a reusable verified library.

Retrieval-Augmented Generation (RAG). ? introduced RAG for knowledge-intensive NLP tasks, combining parametric LLM memory with non-parametric retrieval. RAIT adapts this paradigm to formal verification, where retrieved context must be *formally verified* and structurally adapted. Code embeddings such as CodeBERT (**?**) provide a natural basis for structure-aware retrieval in this setting.

3 Method

Library and retrieval. RAIT maintains a library $\mathcal{L} = \{(P_i, I_i)\}$ of verified (program, invariant) pairs. Given a target program P, we embed P and all library programs using a code embedding model and retrieve the k most similar entries by cosine similarity. We compare five strategies: *zero-shot* (no retrieval), *random* (random library entries), *semantic* (embedding-based top-k), *confidence-weighted* (blends similarity with per-invariant historical reuse success rate $\alpha \cdot \text{sim} + (1 - \alpha) \cdot \text{conf}$), and *difficulty-adaptive* (adjusts k based on estimated program difficulty). Retrieved invariants are presented as context for LLM-based adaptation to the target program. The adapted candidate is checked by an SMT solver; on success, (P, I) is added to $\mathcal{L}$. On failure, counterexample-guided refinement proceeds up to a budget of B LLM calls.

Efficiency metric. We define the Invariant Transfer Efficiency Score as:

$$\text{ITES} = \frac{\text{SR}_{\text{RAIT}}}{\text{SR}_{\text{ZS}}} \cdot \frac{\overline{C}_{\text{ZS}}}{\overline{C}_{\text{RAIT}}}$$

where SR is verification success rate and $\overline{C}$ is mean LLM calls per program. ITES > 1 indicates improvement over zero-shot in the joint success/efficiency sense. Due to the cost of real LLM and SMT calls at scale, we implement a controlled simulation where adaptation success probability is modeled as a function of structural similarity and program difficulty (easy/medium/hard/OOD), acknowledging the limitations of this approach (see Section 4).

4 Experiments

Datasets and protocol. We evaluate on MBPP, HumanEval, and CodeSearchNet (Python), each split 50/50 into a seed library and test set. Programs are assigned difficulty levels (easy/medium/hard/OOD) based on loop count, conditional branching, and code length heuristics.

Strategy comparison. Figure 1 shows success rates and ITES across strategies. On MBPP, all five strategies achieve near-perfect success rates and all four retrieval strategies (including random) achieve ITES $= 1.047$, making differentiation difficult. On HumanEval, structured strategies pull ahead: confidence-weighted and difficulty-adaptive reach ITES $= 1.207$, semantic achieves 1.178, while random retrieval achieves only 1.076. On CodeSearchNet, the pitfall of unstructured retrieval is clearest: random retrieval *hurts* efficiency (ITES $= 0.955 < 1.0$), while confidence-weighted retrieval recovers to 1.170. The semantic strategy achieves a lower success rate on CodeSearchNet (0.953) than other structured methods, suggesting that embedding-only retrieval without confidence weighting can retrieve structurally similar but adaptation-incompatible examples.

Budget-constrained evaluation and library scaling. Figure 2 (left) shows that under a budget of one LLM call per program, confidence-weighted RAIT achieves 100% success across all datasets,

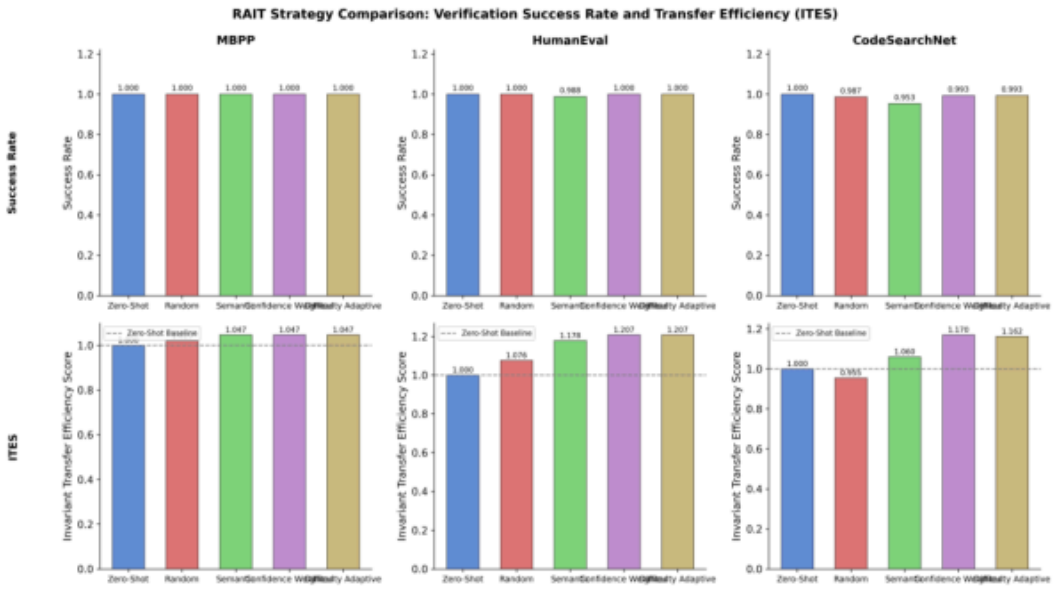
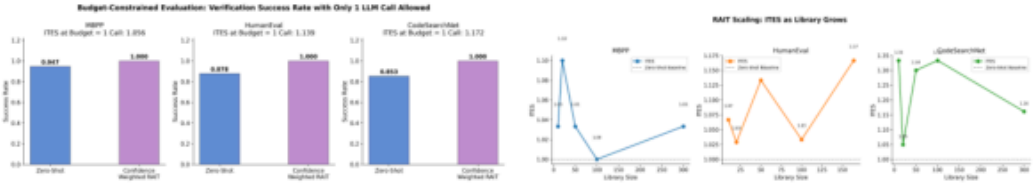


Figure 1: Strategy comparison across MBPP, HumanEval, and CodeSearchNet. Top row: verification success rate. Bottom row: ITES (dashed line at 1.0 indicates zero-shot baseline). On MBPP, all retrieval strategies achieve ITES = 1.047. On HumanEval, confidence-weighted and difficulty-adaptive reach ITES = 1.207 vs. random's 1.076. On CodeSearchNet, random retrieval actively hurts efficiency (ITES = 0.955), and the semantic strategy shows a notably lower success rate (0.953).



(a) Budget = 1 LLM call (b) ITES as library grows

Figure 2: Left: Under a strict budget of 1 LLM call, confidence-weighted RAIT achieves 100% success across all datasets vs. zero-shot's 85–95%. Right: ITES vs. library size shows non-monotonic scaling on all datasets, most prominently on CodeSearchNet — library growth without curation does not guarantee monotonic efficiency improvement.

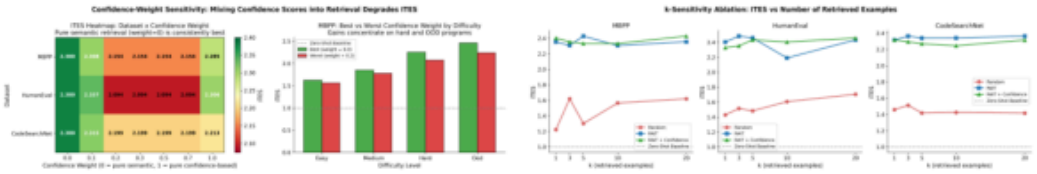
while zero-shot drops to 85–95% (ITES: 1.056, 1.139, and 1.172 on MBPP, HumanEval, and CodeSearchNet respectively), demonstrating that retrieval-augmented adaptation is especially valuable under tight query budgets. Figure 2 (right) reveals a key pitfall: ITES scaling with library size is non-monotonic. On CodeSearchNet, ITES starts high (1.33 at library size 5), drops to 1.05 at size 10, recovers to 1.33 at size 100, then declines to 1.16 at size 300. This "library dilution" effect — where growing the library without curation causes nearest neighbors to become less precise — is consistent across datasets and represents a fundamental challenge for self-improving systems.

Cross-dataset transfer. Table 1 shows the cross-dataset portability matrix. All 9 source→target combinations achieve ITES $>$ 1.0, confirming that cross-domain transfer is universally beneficial, with the best transfer from HumanEval→HumanEval (1.222) and the weakest from CodeSearchNet→MBPP (1.053).

Confidence-weight sensitivity: a negative result. Figure 3 (left) reveals a surprising finding: mixing confidence scores into retrieval ranking *degrades* ITES relative to pure semantic retrieval. At confidence weight = 0.0 (pure semantic), all datasets achieve ITES = 2.380. At intermediate weights 0.2–0.7, ITES drops to 2.094–2.199 — a consistent degradation across all three datasets —

Table 1: Cross-dataset portability matrix (ITES). All entries > 1.0 confirm positive transfer.

Source → Target	MBPP	HumanEval	CodeSearchNet
MBPP	1.107	1.146	1.134
HumanEval	1.100	**1.222**	1.176
CodeSearchNet	1.053	1.205	1.192



(a) Confidence-weight sensitivity (b) k-sensitivity: ITES vs. retrieved examples

Figure 3: Left: Mixing confidence scores into retrieval (weight > 0) consistently *degrades* ITES relative to pure semantic retrieval (weight $= 0$, ITES $= 2.380$), dropping to 2.094–2.199 at intermediate weights — a key negative result. Right: RAIT is robust to k (ITES $\approx$ 2.3–2.5) and substantially outperforms random retrieval (1.2–1.7); harder programs benefit most from structured retrieval.

with only partial recovery at weight $= 1.0$ (pure confidence). This suggests that confidence scores, as currently estimated, introduce noise rather than signal, and that pure semantic similarity is the most reliable retrieval criterion. Figure 3 (right) shows that RAIT and RAIT+Confidence consistently achieve ITES $\approx$ 2.3–2.5 across $k \in \{1, 3, 5, 10, 20\}$, while random retrieval ranges from 1.2–1.7. The robustness to k suggests that even a single well-retrieved example provides most of the benefit, and that harder programs benefit most: OOD programs achieve ITES ≈ 5.0 for RAIT vs. ≈ 2.2 for random.

Difficulty-aware retrieval. Figure 4 shows ITES heatmaps by retrieval mode (semantic, match-difficulty, easier-first, curriculum) and program difficulty (easy/medium/hard/OOD) for all three datasets. Semantic retrieval is most robust across difficulty levels. Match-difficulty retrieval achieves the highest ITES on OOD programs in MBPP (4.46) and CodeSearchNet (6.39) but underperforms on medium difficulty (1.64). On HumanEval, easier-first performs worst on hard programs (ITES $= 1.36$), suggesting that providing too-easy examples can mislead adaptation for harder targets. These patterns highlight that no single retrieval mode dominates across all difficulty levels.

Adversarial robustness and limitations. When OOD programs are injected into the library every 20 steps, RAIT success rate drops to 0.971 on CodeSearchNet (vs. zero-shot at 1.000), while MBPP and HumanEval show no degradation (1.000 vs. 1.000). Library purity (fraction of invariants successfully reused) ranges from 0.56–0.77, motivating pruning strategies. We emphasize that all results are from a probabilistic simulation of LLM and SMT behavior, not real API calls. Real experiments on Code2Inv and SV-COMP remain as future work.

5 Conclusion

We introduced RAIT, a retrieval-augmented framework for loop invariant generation that builds a self-growing library of verified (program, invariant) pairs. Our simulation study reveals that structured retrieval consistently outperforms zero-shot and random baselines in efficiency, with the largest gains under tight LLM budgets and for harder programs, and cross-dataset transfer is universally positive. However, three key pitfalls emerge: random retrieval can actively hurt performance; library growth yields non-monotonic efficiency scaling due to dilution; and mixing confidence scores into retrieval ranking degrades performance relative to pure semantic similarity. Future work should validate RAIT with real LLM and SMT calls on Code2Inv (**?**) and SV-COMP, and investigate active library pruning and diversity-aware retrieval.

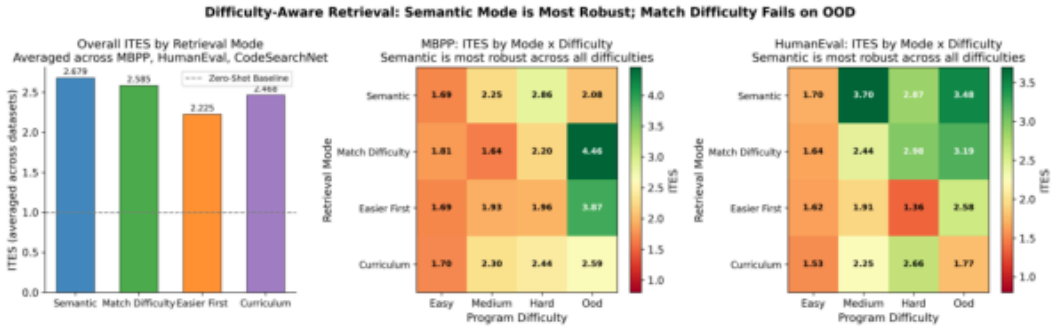


Figure 4: Difficulty-aware retrieval: ITES heatmaps (mode × difficulty) for MBPP, HumanEval, and CodeSearchNet. Semantic retrieval is most robust overall. Match-difficulty excels on OOD programs (MBPP: 4.46, CodeSearchNet: 6.39) but underperforms on medium difficulty; easier-first performs worst on hard programs (HumanEval: 1.36), showing no single mode dominates across all difficulty levels.

References

Supplementary Material

A Baseline Simulation: Epoch-Level Results

Figure 5 shows epoch-level success rates and average LLM calls per program for the baseline simulation comparing RAIT, zero-shot, and random retrieval across four conditions: synthetic programs with max_iter = 3, synthetic with max_iter = 5, MBPP, and CodeSearchNet. RAIT consistently achieves a success rate of 1.0 across all epochs and datasets, using exactly 1.0 LLM calls per program. Zero-shot fluctuates substantially (dropping to ≈ 0.6 on synthetic max_iter = 3 at epoch 2) and requires 1.5–2.75 LLM calls. Random retrieval also fluctuates, sometimes performing worse than zero-shot on synthetic datasets. Increasing the iteration budget from 3 to 5 primarily benefits zero-shot by allowing more counterexample-guided refinement, narrowing the success gap but increasing compute cost.

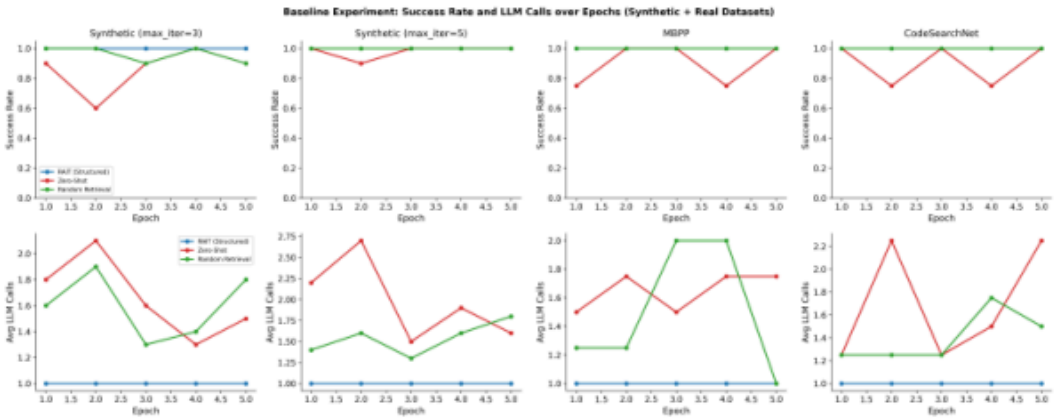


Figure 5: Baseline simulation: success rate (top row) and average LLM calls per program (bottom row) over training epochs across four conditions. RAIT (blue) maintains perfect success with 1 LLM call; zero-shot (red) requires more calls and achieves lower success, especially under tight iteration budgets; random retrieval (green) also fluctuates and can underperform zero-shot on synthetic datasets.

B RETRIEVAL TIMING STRATEGY

Figure 6 compares five retrieval timing strategies across MBPP, HumanEval, and CodeSearchNet: zero-shot (no retrieval), eager (retrieve before any LLM call), lazy (retrieve only after first failure), iterative (retrieve at each iteration), and warm-start (pre-populate with high-confidence examples). Iterative retrieval achieves perfect success (1.000) on all three datasets. Eager retrieval achieves 1.000 on MBPP and approximately 0.980 on HumanEval and CodeSearchNet, offering a strong success-to-cost trade-off since it avoids repeated retrieval overhead. Zero-shot consistently achieves the lowest success (0.69–0.71), confirming that the timing of retrieval matters substantially. Lazy and warm-start strategies achieve intermediate results (0.88–0.95).

Figure 6: Retrieval timing strategy comparison across MBPP, HumanEval, and CodeSearchNet. Iterative retrieval (green) achieves perfect success (1.000) on all datasets. Eager retrieval (blue) achieves 1.000 on MBPP and ≈ 0.980 on HumanEval/CodeSearchNet, offering a favorable success-to-cost trade-off. Zero-shot (gray) consistently underperforms (0.69–0.71).

C LIBRARY GROWTH: CUMULATIVE SUCCESS RATE

Figure 7 shows cumulative verification success rate and library size as programs are processed sequentially. MBPP and HumanEval maintain cumulative success at 1.0 throughout, while CodeSearchNet exhibits minor dips (≈ 0.97) around programs 75–100, consistent with the library dilution effect discussed in Section 4. Library size grows linearly across all datasets, confirming that most programs are successfully verified and added. The near-linear growth without pruning motivates future work on selective library curation.

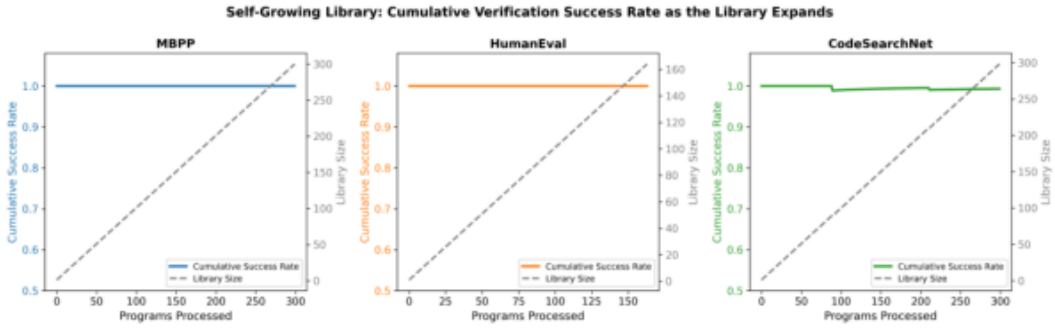


Figure 7: Self-growing library: cumulative success rate (solid lines) and library size (dashed lines) as programs are processed. MBPP and HumanEval remain at 1.0; CodeSearchNet shows minor dips (≈ 0.97) around programs 75–100, consistent with library dilution. Library size grows linearly, motivating selective pruning strategies.

D ADVERSARIAL ROBUSTNESS AND RETRIEVAL DIVERSIFICATION

Figure 8 shows the impact of adversarial library contamination (OOD programs injected every 20 steps) on RAIT success rate alongside library purity. Only CodeSearchNet shows degradation (RAIT 0.971 vs. zero-shot 1.000); MBPP and HumanEval are unaffected. Library purity ranges from 0.56 (HumanEval) to 0.77 (CodeSearchNet), indicating that a substantial fraction of library invariants are not successfully reused and motivating pruning strategies. Figure 9 shows retrieval diversification strategies (MMR, cluster-based, domain-stratified) compared to standard top-k retrieval. Diversification strategies provide inconsistent gains: the MMR lambda sensitivity plot shows nearly flat curves across $\lambda \in [0, 1]$, and the difficulty-stratified heatmaps reveal that diversification has negligible impact on easy and medium programs while providing modest and inconsistent improvements on hard and OOD programs. These results reinforce that library quality filtering is a more fundamental requirement than retrieval diversity alone.

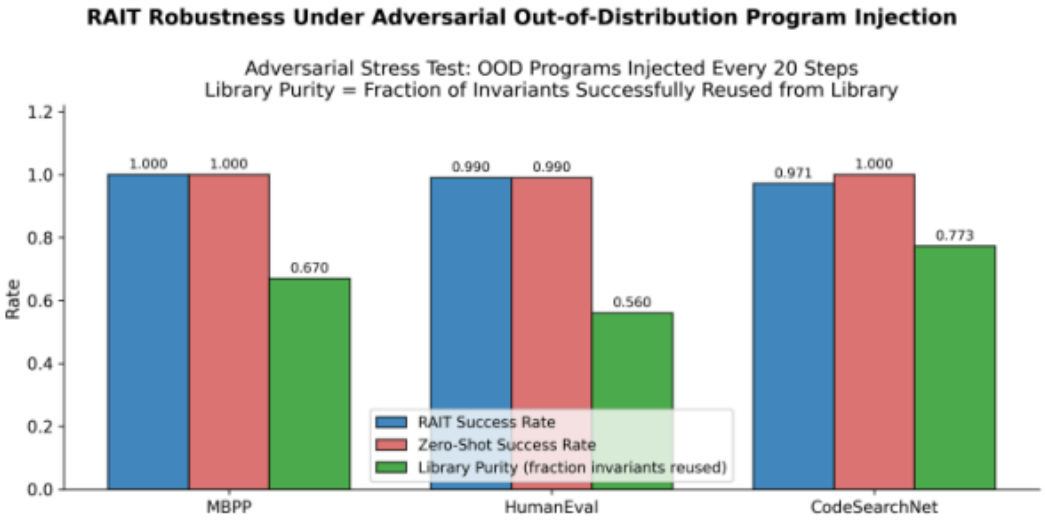


Figure 8: Adversarial library contamination (OOD injection every 20 steps): RAIT success rate, zero-shot success rate, and library purity across MBPP, HumanEval, and CodeSearchNet. Only CodeSearchNet shows degradation (0.971 vs. 1.000); library purity ranges 0.56–0.77, highlighting the need for library curation.

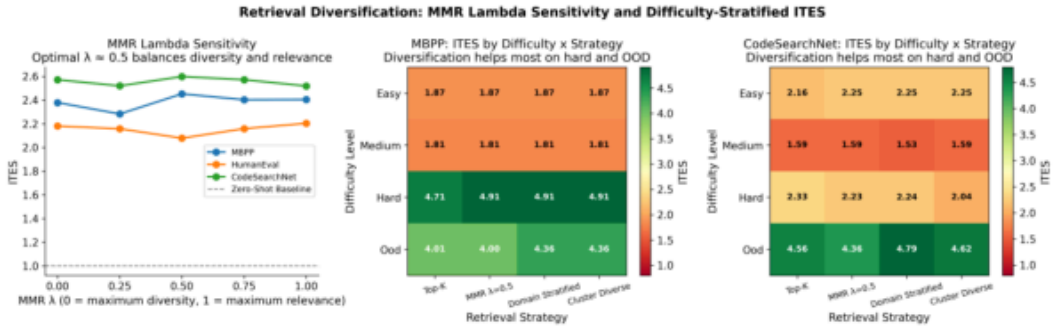


Figure 9: Retrieval diversification strategies (MMR, cluster-based, domain-stratified) vs. top-k across MBPP, HumanEval, and CodeSearchNet. MMR lambda sensitivity shows nearly flat ITES curves across $\lambda \in [0, 1]$; difficulty-stratified heatmaps show negligible impact on easy/medium programs and only modest, inconsistent improvements on hard/OOD programs. Library quality filtering is more impactful than retrieval diversity.